

4+1 view model comprises of five views in total and describes various perspectives that are involved in making of any SW product.

1. LOGICAL VIEW: This view shows the objects and the connections between them. It gives a generic idea of what the system aims to do. It considers all the Functional requirements listed down by the Stakeholder (Client). The targeted audience of this view are the end users, analysts and designers. The UML diagram used to represent this view is class & object diagram.
  2. PROCESS VIEW: This view shows the sequence and process workflows. It is directly linked to the performance of system on runtime. It considers all the Non-functional requirements stated in ~~SRS~~ SRS. The targeted audience are the developers and Quality Assurance team. The UML diagram used to represent this view is activity <sup>state</sup> sequence and communication diagrams.
  3. DEVELOPMENT VIEW: This view shows the building block view of the software and describes the organization of modules. It considers the reusable component constraints and software modules. Targeted audience are developers. UML diagram used to represent this view is component diagram.
  4. DEPLOYMENT VIEW: This explains installation / configuration and development of SW and the hardware involved. It considers the NFRs regarding underlying hardware. Targeted audience are all stakeholders whose work is associated with installation for e.g system engineers. UML diagram used is deployment diagram.
  5. USE CASE VIEW: This view validates the performance of system from a user's perspective. It considers system consistency and ensures system is a whole workable unit. Targeted audience are end users. UML diagram used is use case diagram.
- This is called 4+1 view model instead of 5 view model because the last view that is use case view validates the previous 4 views. When user performs this validation, it is done with respect to the other 4. The correct adaption to the first



four views can only lead to the 5<sup>th</sup> view being validated.

Q2] Microservice architecture has following characteristics

- loose coupling
- local database for each service
- scalability of entire system.
- use of APIs to communicate between services and their D/B.
- network overheads are high due to more time spent in data fetching.
- high latency.
- individual deployment of services is possible
- different frameworks and languages can be used
- maintaining and updating the system is easier

Real life Examples → Netflix and Amazon.

These both have services that are partitioned with each service getting its relevant data from its own database.

Crashing or bug <sup>issue</sup> in any one service does not affect the rest of the services. However the time taken to fetch data is higher as compared to monolithic because the request is routed to a different network via API.

Q3] TECHNICAL REQUIREMENT FOR DISASTER MANAGEMENT AND RESCUE.

1. System should be able to send alerts in case of expected disasters.
2. Maps should be made available by integrating google maps to show location of shelter homes to users, location of victims to government authorities & hospitals.
3. In case of aid distribution, tracking aid should be made available. 3.5
4. Payment should be facilitated and verified using API gateways.
5. A camera should be integrated within system to show the intensity of disaster or to seek virtual help.
6. Smooth communication either via call or messaging should be supported within the system.
7. Unattended victims should be displayed on the maps with red flags and attended/saved people should be displayed on the map with green flags to avoid confusion between rescue teams.



6. Based on location of disaster, the system should give estimated time and suggest alternative routes for better and faster management.

9. Medical team should be kept in loop via <sup>app</sup> in+chat messaging

10. Rescue teams should be able to keep track of medical and aid kits sent. A weekly report could be generated by system.

SCENARIO VIEW  
DIAGRAM.

